ASSIGNMENT 4

Relational Database vs. NoSQL Database

Comparative report with real-world examples, applied to the Online Shopping System

1. Introduction

A **relational database (RDBMS)** stores data in tables of rows and columns with a fixed schema, links tables through primary and foreign keys, and is queried with SQL. **NoSQL** ("not only SQL") is an umbrella term for databases that use other data models – documents, key–value pairs, wide columns or graphs – usually with a flexible schema and a design aimed at horizontal scaling. Neither is "better"; each makes different trade-offs. This report compares them on the four requested criteria and then explains which technology suits which part of our Online Shopping System.

2. Types of NoSQL Databases

Type	Data model	Examples	Typical strength
Document	JSON-like documents grouped in collections	MongoDB, Couchbase	Flexible, nested records such as product catalogues
Key–value	Unique key → opaque value	Redis, Amazon DynamoDB	Very fast lookups: sessions, carts, caching
Wide-column	Rows with dynamic column families, distributed	Apache Cassandra, HBase	Huge write volumes, time-series, activity logs
Graph	Nodes and edges	Neo4j, Amazon Neptune	Relationship traversal: recommendations, fraud rings

3. Comparison on the Required Criteria

3.1 Scalability

Relational: traditionally scales *vertically* – a bigger server with more CPU, RAM and faster disks. Read capacity can be scaled with replicas, and large tables can be partitioned or sharded, but sharding a relational database that relies on joins and multi-row transactions is complex and often needs application-level logic. **NoSQL:** most systems were built to scale *horizontally* – data is automatically partitioned (sharded) across many commodity nodes and more nodes are added as load grows. Because their data model avoids cross-record joins, adding nodes increases capacity almost linearly.

3.2 Consistency

Relational: provides **ACID** transactions – *Atomicity*, *Consistency*, *Isolation*, *Durability*. A checkout that inserts an order, its items and a payment either completes fully or not at all, and constraints (PK, FK, UNIQUE, CHECK) guarantee valid data. **NoSQL:** many systems follow the **BASE** philosophy – *Basically Available, Soft state, Eventually consistent*. By the **CAP theorem**, when a network partition occurs a distributed system must choose between consistency and availability; systems such as Cassandra and DynamoDB favour availability and offer tunable or eventual consistency, while MongoDB and others offer strong consistency per document and, in newer versions, multi-document transactions at some performance cost.

3.3 Performance

Relational: excellent for complex queries – joins, aggregation, sorting and ad-hoc reporting – thanks to mature query optimizers and indexes; performance drops when very large joins span huge tables or when a single node becomes the write bottleneck. **NoSQL:** excellent for simple, predictable access patterns (get/put by key, read a whole document) with very low latency and very high write throughput, because related data is stored together (denormalized) and requests are spread across nodes. Ad-hoc analytics and cross-collection joins are weaker or must be handled by the application.

3.4 Use cases

Relational fits structured data with clear relationships and strict correctness: banking and payments, order management, inventory, ERP/CRM, reservation systems, and reporting. **NoSQL** fits very large or rapidly changing data with flexible structure and massive scale: product catalogues with varying attributes, shopping sessions, user activity feeds, IoT and time-series data, content management, caching, and recommendation graphs.

3.5 Summary table

Aspect	Relational (RDBMS)	NoSQL
Data model	Tables, rows, columns; fixed schema	Documents, key–value, wide-column or graph; flexible schema
Query language	Standard SQL, powerful joins and aggregates	Database-specific APIs/query languages; limited joins
Scalability	Mainly vertical; sharding is complex	Designed for horizontal scale-out
Consistency	Strong ACID transactions	Often BASE / eventual; tunable; some offer ACID per document
Integrity rules	PK, FK, UNIQUE, CHECK enforced by the DBMS	Mostly enforced in the application
Schema change	Needs ALTER TABLE / migration	Add new fields at any time
Performance strength	Complex queries, reporting, multi-table transactions	Fast key/document access, massive write throughput
Redundancy	Normalized – minimal duplication	Denormalized – duplication accepted for speed
Best for	Orders, payments, inventory, finance	Catalogue, sessions, carts, logs, recommendations
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Redis, Cassandra, DynamoDB, Neo4j

4. Why Organizations Choose Different Database Technologies

Technology is chosen by requirements, not fashion. The usual deciding factors are:

- **Data structure and relationships** – highly interconnected, regular data → relational; varied, nested or evolving records → document; relationship-heavy → graph.
- **Correctness requirements** – money movement, stock counts and legal records need ACID guarantees; a "like" counter or a feed can tolerate being a few seconds out of date.
- **Scale and traffic pattern** – millions of writes per second across regions favour distributed NoSQL; moderate, query-rich workloads run perfectly well on a single relational server.
- **Query flexibility** – business reporting needs ad-hoc SQL; fixed access patterns can be served by key lookups.
- **Speed of change** – start-ups with rapidly changing data models value schema flexibility; regulated enterprises value enforced schemas.
- **Skills, tooling and cost** – SQL skills are widespread and tooling is mature; some NoSQL systems reduce operational cost at very large scale through commodity hardware.
- **Availability needs** – services that must keep accepting writes during failures accept eventual consistency in return.

Because no single engine is best at everything, large organizations commonly adopt **polyglot persistence** – using several database types, each for the job it does best.

5. Real-World Examples

Organization / system	Technology	Why it was chosen
Banks and payment systems (e.g. core banking running on Oracle, IBM Db2 or similar RDBMS)	Relational	A money transfer must debit one account and credit another atomically; ACID transactions and strict constraints are non-negotiable.
Amazon – shopping cart (Dynamo, described in the 2007 Dynamo paper; later offered as Amazon DynamoDB)	Key–value NoSQL	The cart must always accept "add item" even during server or network failures, so availability was prioritised over strict consistency; conflicts are merged later.
Netflix and Facebook (Cassandra was originally developed at Facebook and is used at Netflix)	Wide-column NoSQL	Enormous write volumes and multi-region availability that a single relational server could not provide.

Organization / system	Technology	Why it was chosen
Twitter and many web applications using Redis	Key–value / in-memory	Sub-millisecond reads for caching, timelines and session data.
MongoDB in content and catalogue platforms	Document NoSQL	Records with different attributes (a phone vs. a shirt) fit naturally into flexible JSON documents.
Neo4j in recommendation and fraud-detection systems	Graph NoSQL	Queries such as "customers who bought X also bought Y" or detecting linked suspicious accounts are fast graph traversals.

6. Application to the Online Shopping System

6.1 Why the project uses a relational database

The core of our system – customers, addresses, orders, order items, payments and shipments – is highly structured and interconnected, and it must be **correct**: an order must never reference a missing customer, a payment must belong to exactly one order, and a rating must lie between 1 and 5. The relational model provides exactly this through primary keys, foreign keys, UNIQUE and CHECK constraints (Assignments 5 and 6), and SQL joins and aggregates give the reports required in Assignments 8 and 9. Order placement is a multi-table transaction, which is the textbook strength of ACID.

6.2 Where NoSQL could complement it at larger scale

Part of the system	Best-fit technology	Reason
Customers, Orders, Order_Items, Payments, Shipments	Relational (MySQL)	ACID transactions and integrity constraints
Product catalogue with varying attributes (specifications, images, variants)	Document (MongoDB)	Each product is a self-contained JSON document with different fields
Active carts and login sessions	Key–value (Redis)	Very low latency, temporary data keyed by customer
Clickstream and activity logs	Wide-column (Cassandra)	Very high write volume, append-only
"Customers also bought" recommendations	Graph (Neo4j)	Fast traversal of customer–product relationships

6.3 Same data in both models

A product and its reviews as **relational rows** (normalized, joined at query time):

Relational form (illustrative values)

```
Products:  (2, 1, 'Rockerz 450 Bluetooth Headphones', 1499.00, 120, 'boAt')
Reviews:   (7, 2, 4, 5, 'Excellent quality, totally worth the price.')
           (19, 2, 9, 4, 'Good value for money, minor packaging issue.')
-- SELECT p.product_name, r.rating FROM Products p JOIN Reviews r ON r.product_id = p.product_id;
```

The same product as a **MongoDB document** (denormalized, read in one lookup):

Document form

```
{
  "_id": 2,
  "name": "Rockerz 450 Bluetooth Headphones",
  "brand": "boAt",
  "category": "Electronics",
  "price": 1499.00,
  "stock": 120,
  "specs": { "type": "On-ear", "battery_hours": 15, "bluetooth": "5.0" },
  "reviews": [
    { "customer_id": 4, "rating": 5, "comment": "Excellent quality, totally worth the price." },
    { "customer_id": 9, "rating": 4, "comment": "Good value for money, minor packaging issue." }
  ]
}
```

The document is faster to display on a product page and allows each product to have different "specs", but the reviewer details and product price are now duplicated wherever they are embedded – exactly the redundancy and update-anomaly risk studied in Assignment 3. This is the central trade-off between the two worlds.

7. Conclusion

Relational databases offer strong consistency, enforced integrity and powerful SQL, and are the right foundation for transactional data such as orders and payments. NoSQL databases offer horizontal scalability, flexible schemas and very

fast simple access, which suits catalogues, sessions, logs and recommendations at large scale. Organizations choose according to data shape, consistency needs, scale and query patterns – and mature platforms often combine both. For this project, a relational design is the correct choice, with NoSQL stores identified as optional additions if traffic and catalogue complexity grow.